



Mariposas de Cuba — app nativa de iOS

Proyecto Xcode en SwiftUI · rama [claude/ios-native-app-9n0hlj](#)

Esta rama trae un puerto real de **Mariposas de Cuba** a SwiftUI: un proyecto de Xcode que compila a un `.ipa`, como si fuera a subirse a la App Store — no una web disfrazada. Vive en `ios/MariposasDeCuba/`.

La rama también conserva, en `src/`, un rediseño anterior de la versión web con la misma paleta y tipografía (una PWA instalable con "Añadir a pantalla de inicio"). Sigue ahí por si sirve, pero **lo que se documenta aquí es esto**: código Swift real, compilado por Xcode.

1. Qué es esto

Un proyecto de Xcode completo, con SwiftUI y sin storyboards:

```
ios/
├── MariposasDeCuba.xcodeproj/    proyecto de Xcode
├── MariposasDeCuba/
│   ├── App/                     arranque + Info.plist
│   ├── Models/                  Especie, Zona, Avistamiento...
│   ├── Store/                   Almacen (persistencia), filtros
│   ├── Design/                  paleta de colores, tipografía
│   ├── Views/Catalogo/          cuadrícula, filtros
│   ├── Views/Ficha/             ficha de especie y todo lo suyo
│   ├── Resources/               especies.json, zonas.json
│   └── Assets.xcassets/         icono de la app, color de acento
├── generate_project.rb          genera el .xcodeproj
└── ci/elegir_simulador.py      usado por el workflow de CI
```

Dos pantallas, igual que en la web:

- **Catálogo** — `CatalogoView`: cuadrícula de dos columnas con `.searchable()` nativo, control segmentado Láminas/Montados, hoja de filtros y modo de selección manual.
- **Ficha** — `FichaView`: identificación, distribución (mapa de Cuba con las coordenadas reales de `zonas.json`), ciclo de vida con foto por estadio, avistamientos con mapa propio, galería personal y notas.

Los datos son los mismos dos archivos de siempre, empaquetados tal cual en la app y leídos con `Codable` — no hubo que convertir ni un campo. Lo que anotas tú se guarda con `FileManager` + JSON en el `Documents` de la app, el equivalente nativo del `localStorage` de la versión web.

2. Por qué se ve como una app de Apple de verdad

No hay nada que imitar: es SwiftUI sobre UIKit, así que estos son componentes nativos reales, no CSS parecido:

- `NavigationStack` con título grande que se encoge al hacer scroll.
- `.searchable()` — la barra de búsqueda del sistema, con su cancelar y su animación propios.
- `.sheet()` para las hojas modales: el asa, el desenfoque de fondo y el gesto de cerrar arrastrando los trae UIKit de fábrica.
- `Picker(.segmented)` — el control segmentado real.
- `Form` agrupado para "Anotar avistamiento".
- `PhotosPicker`, `UIImagePickerController` y `CoreLocation` para fotos y GPS.

La paleta es la misma que se afinó para la web: turquesa de *Eumaeus atala* como acento (coincide con el `systemTeal` de Apple), colores dinámicos de verdad (`UIColor { traits in ... }` en `Design/Paleta.swift`) que siguen el modo claro/oscuro del teléfono en tiempo real. Los nombres científicos usan New York (`design: .serif`) — nada que empaquetar.

3. Qué se simplificó en esta primera versión

Portar 4000 líneas de React a Swift con paridad exacta en cada gesto es un proyecto de semanas. Esta primera versión prioriza que se vea, se sienta y compile como una app nativa real, con todas las pantallas y datos funcionando. Un recorte consciente:

Mapa de zonas: de toque, no de trazo a mano alzada. La web deja dibujar con el dedo sobre el mapa para elegir zonas (detección de solape trazo ↔ polígono a medida). La app nativa usa las mismas coordenadas reales de `zonas.json` y se puede tocar una zona para elegirla, pero no "pintar" un área con el dedo. Mismo dato, mismo mapa, gesto más simple.

Todo lo demás — catálogo, filtros, ficha completa, avistamientos con GPS y foto, ciclo de vida, galería, notas, selección manual — está completo y funcionando.

4. Cómo se compila (no hace falta tu Mac)

Xcode solo existe para macOS, así que el proyecto se separó en dos partes:

1. **Generar el `.xcodproj`** no necesita macOS: `generate_project.rb` usa la gema Ruby `xcodproj` (la misma que usan CocoaPods y fastlane) para escribir el proyecto directamente.
2. **Compilarlo** sí necesita Xcode/macOS de verdad — para eso está `.github/workflows/build-ios.yml`, que corre en un runner de macOS de **GitHub Actions** (gratis para repositorios públicos). Ese runner compila el proyecto sin firmar, empaqueta el `.ipa`, y además compila una copia para el Simulador, la instala en modo claro y oscuro, y guarda una captura real de cada uno.

Esto resuelve justo la limitación de hardware: **ni tu MacBook de 2014 ni ningún otro Mac necesitan compilar nada** — lo hace el runner de Apple que GitHub presta gratis.

```
cd ios
ruby generate_project.rb      # solo si añadiste o quitaste archivos
```

Luego, en GitHub: pestaña Actions → "Compilar app nativa de iOS" → Run workflow (disponible una vez que este archivo llegue a main; mientras tanto se dispara solo con cada push a esta rama que toque ios/).

5. Cómo ponerla en tu iPhone — sin Xcode, sin iTools, sin 3uTools

Un `.ipa` tiene que estar firmado para que iOS lo abra, incluso en tu propio teléfono. El que produce el CI no está firmado a propósito — firmarlo en GitHub necesitaría meter tu Apple ID en un secreto de CI, y no hace falta: las herramientas de abajo firman el `.ipa` con tu Apple ID **al instalarlo**, el mismo mecanismo gratuito que usa Xcode para "Ejecutar" una app en tu propio dispositivo sin cuenta de desarrollador de pago.

Por qué no iTools/3uTools: su función de "instalar IPA" depende de firmar con un certificado, y en iOS moderno sin jailbreak eso solo funciona de forma fiable con herramientas que hablan directamente con los servicios de desarrollador de Apple. La recomendación real hoy es **Sideloadly** o **AltStore**.

Paso 1 — descarga el `.ipa`

1. En GitHub, pestaña **Actions** del repositorio.
2. Abre la ejecución más reciente de "Compilar app nativa de iOS".
3. Baja hasta **Artifacts** y descarga `MariposasDeCuba-ipa`.

Paso 2 — instala con Sideloadly (recomendado)

Corre en **Windows o macOS** — tu Mac de 2014, aunque no pueda con Xcode moderno, sí puede con Sideloadly (pide mucho menos). Si tampoco, sirve cualquier PC con Windows a mano.

1. Descarga Sideloadly desde **sideloadly.io** e instálalo (en Windows hace falta también iTunes instalado, por los drivers — no hay que abrirlo).
2. Conecta el iPhone por cable.
3. Abre Sideloadly, arrastra el **.ipa** a la ventana.
4. Escribe tu Apple ID (uno normal y gratis) cuando lo pida — se usa un momento para firmar, no se guarda la contraseña.
5. Toca **Start**. El icono aparece en la pantalla de inicio en unos minutos.
6. La primera vez: **Ajustes** → **General** → **VPN y gestión de dispositivos** → **(tu Apple ID)** → **Confiar**.

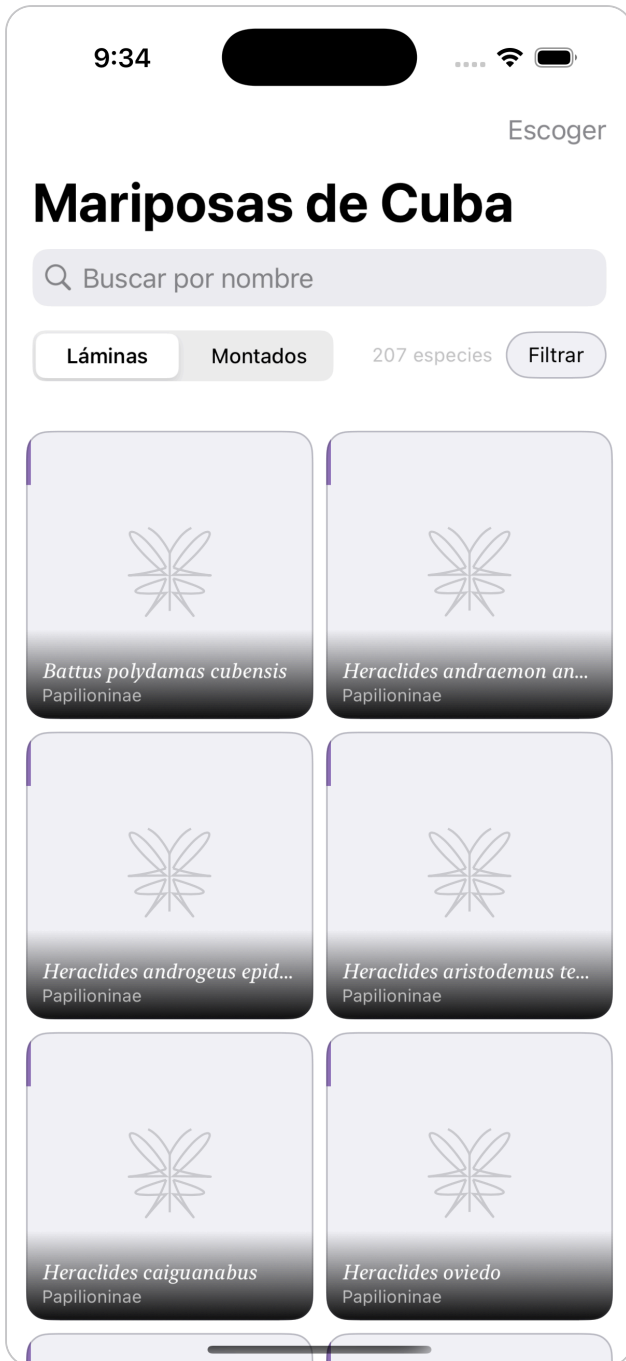
Con un Apple ID gratis, la app deja de abrir a los 7 días (límite de Apple, no de Sideloadly) — se reinstala con el mismo .ipa en un minuto. AltStore puede renovar la firma solo por Wi-Fi sin repetir el proceso.

Sobre iTools y 3uTools

Siguen sirviendo para fotos, backups, explorar archivos. Lo único que no resuelven de forma fiable es este paso concreto de instalar un **.ipa** propio.

6. Capturas del simulador

Reales, tomadas por el propio CI en un Simulador de iPhone — no un mockup.



Catálogo — modo claro



Catálogo — modo oscuro

7. Si quieres seguir desarrollando

Con un Mac que corra una versión de Xcode razonablemente reciente (15+):

```
cd ios
ruby generate_project.rb # si falta el gem: gem install xcodeproj
open MariposasDeCuba.xcodeproj
```

Añadir las fotos reales del catálogo: en Xcode, arrastra una carpeta `img` (misma estructura que `public/img/` en la web) a `MariposasDeCuba/Resources`, eligiendo "Create folder references" (icono azul, no amarillo) para que `Lamina.swift` las encuentre por nombre de archivo.

Todo lo de `README.md` sobre el formato de los datos sigue aplicando igual — el catálogo es el mismo JSON en los dos lados.